BÀI HƯỚNG DẪN THỰC HÀNH

XÂY DỰNG BLOCKCHAIN WALLET CHO ERC20 VÀ ERC721

Sản phẩm: ERC20 MoolahCoin + ERC721 Condos + triển khai trên Ganache + kết nối MetaMask/React

1. Mục tiêu bài thực hành

- Phân biệt fungible token và non-fungible token.
- Tạo một ERC20 token bằng Solidity.
- Tạo một ERC721 NFT có metadata.
- Sử dụng OpenZeppelin để xây dựng smart contract.
- Compile và deploy smart contract bằng Truffle.
- Sử dụng Ganache làm Ethereum blockchain cục bộ.
- Kết nối MetaMask với Ganache.
- Gọi smart contract từ ứng dụng React/Web3.
- Thực hiện các thao tác mint, transfer và approve.

MoolahCoin được dùng làm fungible payment token và Condos là NFT đại diện quyền sở hữu căn hộ.

2. Chuẩn bị môi trường

2.1. Công cụ

Công cụ	Mục đích
Node.js + npm	Môi trường JavaScript và quản lý package
Ganache	Local Ethereum blockchain
Truffle	Compile / migrate / deploy smart contract
MetaMask	Quản lý account và ký transaction
Solidity	Ngôn ngữ smart contract
OpenZeppelin	Thư viện triển khai ERC20/ERC721
ReactJS	Wallet frontend
Web3.js	Kết nối React với Ethereum

Lưu ý: Tối thiểu Solidity 0.5.2, OpenZeppelin 2.2.0, Node 13.0.1 và npm 6.12.0.

2.2. Tạo project

```
mkdir tokenwallet
cd tokenwallet

npm init -y
npm install openzeppelin-solidity@2.2.0
```

```
npm install -g truffle
truffle init
```

Sau khi khởi tạo, quan sát cấu trúc:

```
tokenwallet/
├── contracts/
├── migrations/
├── test/
└── truffle-config.js
```

Câu hỏi cho sinh viên: Chụp ảnh kết quả project wallet vừa tạo

3. Phần B – Tạo ERC20 Token

Bước 1. Tạo MoolahCoin.sol
Trong thư mục contracts/, tạo file MoolahCoin.sol:

```
pragma solidity ^0.5.2;

import "openzeppelin-solidity/contracts/token/ERC20/ERC20Detailed.sol";
import "openzeppelin-solidity/contracts/token/ERC20/ERC20Capped.sol";
import "openzeppelin-solidity/contracts/ownership/Ownable.sol";

contract MoolahCoin is ERC20Detailed, ERC20Capped, Ownable {
  constructor()
    ERC20Detailed("Moolah Coin", "MC", 4)
    ERC20Capped(10000000000)
    payable public
  {
  }
}
```

Ý nghĩa các thành phần:

- ERC20Detailed: khai báo name, symbol và decimals.
- ERC20Capped: giới hạn tổng nguồn cung và hỗ trợ cơ chế mint.
- Ownable: cung cấp cơ chế kiểm soát quyền sở hữu/access control.

Câu hỏi cho sinh viên: xác định Token name, Token symbol, Decimals và Maximum supply.

4. Phần C – Tạo ERC721 NFT

Bước 2. Tạo Condos.sol

```
pragma solidity ^0.5.2;

import "openzeppelin-solidity/contracts/token/ERC721/ERC721Metadata.sol";

contract Condos is ERC721Metadata {

    uint32 public Id;
    mapping(address => uint32) public MDTrack;

    constructor()
        ERC721Metadata("Condos Token", "CONDO")
        payable public
    {
        Id = 0;
    }

    function createNFT(address receiver, string calldata metadata)
        external returns (uint32)
    {
        Id++;
        _mint(receiver, Id);
        _setTokenURI(Id, metadata);
        MDTrack[receiver] = Id;
        return Id;
    }

    function transferNFT(
        address sender,
        address receiver,
        uint32 transId,
        string calldata metadata
    )
        external
    {
        _transferFrom(sender, receiver, transId);
        _setTokenURI(transId, metadata);
        delete MDTrack[sender];
```

```
        MDTrack[receiver] = Id;
    }
}
```

Bước 3. Phân tích createNFT()

Lệnh	Ý nghĩa
Id++;	Tạo Token ID mới.
_mint(receiver, Id);	Mint NFT cho receiver.
_setTokenURI(Id, metadata);	Gắn metadata với NFT.
MDTrack[receiver] = Id;	Ghi nhận Token ID mà địa chỉ đang giữ trong cấu trúc của ví dụ.

Luồng mint NFT:

```
createNFT(Alice, metadata)
        ↓
    Id = Id + 1
        ↓
  _mint(Alice, Id)
        ↓
_setTokenURI(Id, metadata)
        ↓
  NFT belongs to Alice
```

5. Phần D – Cấu hình Ganache và Truffle

Bước 4. Khởi động Ganache

Khởi động Ganache và cấu hình:

Host: 127.0.0.1
Port: 8545

Sinh viên ghi lại:

Account 1 address: _____
Account 2 address: _____
Account 1 balance: _____

Bước 5. Cấu hình Truffle

Mở truffle-config.js và cấu hình development network:

```
module.exports = {
   networks: {
      development: {
         host: "127.0.0.1",
```

```
            port: 8545,
            network_id: "*"
        }
    },
    compilers: {
        solc: {
            version: "0.5.2"
        }
    }
};
```

Câu hỏi: Chụp ảnh kết quả truffle

6. Phần E – Compile Smart Contract

Bước 6. Compile

```
truffle console


truffle(development)> compile
```

Hoặc:

```
truffle compile
```

Nếu thành công, kiểm tra thư mục build/contracts/:

```
build/
└── contracts/
    ├── MoolahCoin.json
    └── Condos.json
```

Mở file JSON và tìm hai thành phần quan trọng: abi và bytecode.

Câu hỏi: ABI dùng để làm gì? Chụp kết quả compile

7. Phần F – Deploy Smart Contract

Bước 7. Tạo migration

Tạo migration trong thư mục migrations/ để triển khai MoolahCoin và Condos. Migration có nhiệm vụ deploy contract, xử lý dependency, khởi tạo constructor và quản lý deployment.

Bước 8. Deploy

```
truffle migrate
```

Ghi lại địa chỉ contract:

MoolahCoin address:

_____

Condos address:

_____

8. Phần G – Kết nối MetaMask với Ganache

Bước 9. Chọn local network

Trong MetaMask, chọn local network tương ứng với localhost:8545.

Bước 10. Import account

Lấy private key của một tài khoản test trong Ganache và import vào MetaMask.

```
Ganache Account
    ↓ Private Key
  MetaMask
    ↓
React Wallet
```

Lưu ý an toàn: chỉ dùng private key của tài khoản test Ganache trong bài lab. Không chia sẻ hoặc nhập private key của ví thật.

9. Phần H – Tạo React Wallet

Bước 11. Tạo ứng dụng

```
npx create-react-app tokenwallet
cd tokenwallet
npm install
```

Bước 12. Tổ chức thư mục

```
src/
├── Components/
├── Token/
│   ├── all20.js
│   ├── all721.js
│   ├── MoolahCoin.js
│   └── Condos.js
└── Contracts/
```

Copy MoolahCoin.json và Condos.json vào src/Contracts/.

10. Phần I – Đăng ký Token với Wallet

Bước 13. ERC20 token list – all20.js

```
import MoolahCoin from './MoolahCoin';

const Tokens20 = [
   MoolahCoin
];

export default Tokens20;
```

Bước 14. ERC721 token list – all721.js

```
import Condos from './Condos';

const Tokens721 = [
   Condos
];

export default Tokens721;
```

Câu hỏi: Chụp ảnh kết quả sau khi đăng ký

11. Phần J – Đọc ERC20 Token

Bước 15. Tạo contract object và đọc balance

```
const contract =
   new web3.eth.Contract(
      token.abi,
      token.address
   );

contract.methods
      .balanceOf(account)
      .call();
```

Luồng dữ liệu:

```
MetaMask Account
      ↓
 balanceOf(account)
      ↓
ERC20 Contract
```

↓
Balance
↓
React Wallet

Câu hỏi: balanceOf(account) có làm thay đổi trạng thái blockchain không?

12. Phần K – Đọc NFT

Bước 16. Lấy Token ID và metadata

```
contract.methods
    .MDTrack(account)
    .call();
```

Sau khi nhận tokenId:

```
contract.methods
    .tokenURI(tokenId)
    .call();
```

```
Account
  ↓
MDTrack(account)
  ↓
Token ID
  ↓
tokenURI(tokenId)
  ↓
Metadata
```

13. Phần L – Thử nghiệm Mint và Transfer

Bước 17. Mint ERC721

Ví dụ dữ liệu:

```
Receiver:
0x1234...

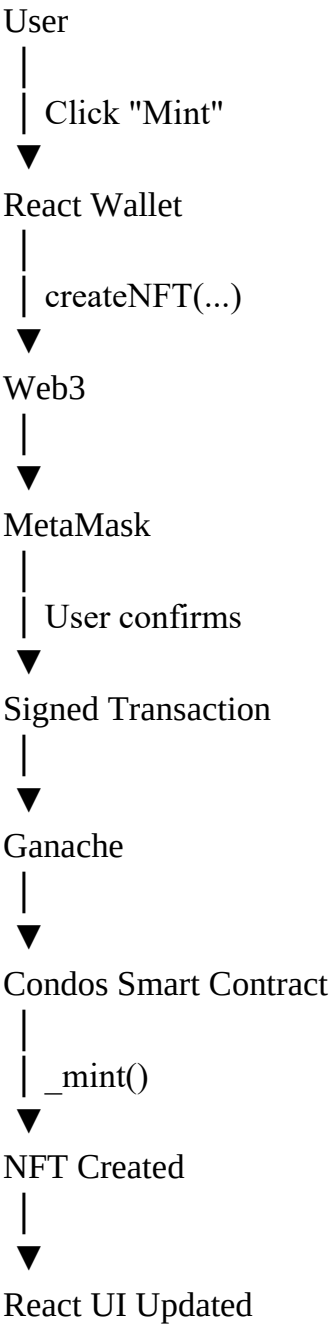
Metadata:
Apartment A - Ho Chi Minh City - 80m2
```

Ứng dụng gọi createNFT(receiver, metadata). MetaMask hiển thị yêu cầu xác nhận transaction. Sau khi transaction được xác nhận, kiểm tra Token ID, owner và metadata.

Bước 18. Transfer
Thực hiện chuyển ERC20 hoặc ERC721 sang một tài khoản Ganache khác. Kiểm tra lại balance/owner sau khi transaction được xác nhận.

14. Phần M – Kiểm tra Transaction Workflow

Sinh viên phân tích toàn bộ quá trình:



15. Yêu cầu nộp bài

#	Minh chứng
1	Screenshot Ganache đang chạy

2	Source code MoolahCoin.sol
3	Source code Condos.sol
4	Kết quả compile thành công
5	Kết quả deploy hai contract
6	Contract address của ERC20
7	Contract address của ERC721
8	MetaMask kết nối local blockchain
9	Wallet hiển thị ERC20 balance
10	Wallet hiển thị NFT
11	Transaction mint NFT thành công
12	Transaction transfer token thành công

PHỤ LỤC A – HƯỚNG DẪN TẢI VÀ CÀI ĐẶT MÔI TRƯỜNG LOCAL

Lưu ý: Nên thực hiện bài lab trên máy cá nhân thay vì Google Colab. Bài thực hành cần trình duyệt + MetaMask tương tác với blockchain cục bộ, nên môi trường local dễ cấu hình và dễ quan sát hơn.

A.1. Tổng quan các công cụ

Công cụ	Vai trò	Cách cài
Node.js + npm	Môi trường JavaScript và quản lý package	Cài Node.js; npm đi kèm
Ganache	Blockchain Ethereum cục bộ	Cài Ganache GUI hoặc dùng bản lưu trữ
Truffle	Compile, migrate, deploy contract	Cài bằng npm
MetaMask	Ví, account và ký transaction	Cài extension trình duyệt
Solidity	Ngôn ngữ smart contract	Compiler do Truffle quản lý
OpenZeppelin	Thư viện ERC20/ERC721	Cài bằng npm trong project
ReactJS	Giao diện wallet	Tạo project bằng npx/npm
Web3.js	React giao tiếp với Ethereum	Cài bằng npm trong project
VS Code	Soạn thảo Solidity/JavaScript	Cài ứng dụng desktop

A.2. Cài Node.js và npm

1. Truy cập trang chính thức: Node.js Download

2. Trên Windows, tải bản LTS phù hợp và chạy bộ cài. Có thể giữ các tùy chọn mặc định.

3. Sau khi cài xong, mở Command Prompt hoặc PowerShell và kiểm tra:

```
node --version
npm --version
```

Nếu cả hai lệnh đều trả về số phiên bản thì Node.js và npm đã được cài thành công.

A.3. Cài Visual Studio Code

1. Truy cập: Visual Studio Code

2. Tải bản Windows và cài đặt. Khuyến nghị bật tùy chọn thêm VS Code vào PATH nếu bộ cài cung cấp.

3. Có thể cài thêm extension hỗ trợ Solidity để tô màu cú pháp và hỗ trợ soạn thảo.

A.4. Cài Ganache

Lưu ý: Ganache cung cấp blockchain để thử nghiệm; Truffle cung cấp công cụ để phát triển và đưa smart contract lên blockchain đó. Ganache và Truffle đã được Consensys sunset; tài liệu chính thức hiện được lưu trữ. Phần này vẫn dùng Ganache.

Tài liệu/phiên bản Ganache lưu trữ: Ganache – Truffle Suite Archive

Sau khi cài và mở Ganache:

1. Chọn QUICKSTART hoặc tạo Workspace mới.
2. Kiểm tra RPC server/port. Bài trong sách sử dụng localhost:8545.
3. Quan sát danh sách account, địa chỉ, balance và private key của các tài khoản test.

    RPC host: 127.0.0.1
    Port: 8545

An toàn: chỉ sử dụng private key của tài khoản test do Ganache tạo. Không dùng private key của ví có tài sản thật.

A.5. Cài Truffle

Sau khi Node.js/npm hoạt động, mở Command Prompt/PowerShell:

```
npm install -g truffle
truffle version
```

Nếu lệnh truffle version hiển thị thông tin Truffle/Solidity/Node thì cài đặt đã thành công.

Ghi chú: Truffle cũng là công cụ legacy/sunset. Có thể dụng Hardhat 3 để thay Truffle + Ganache. Hardhat 3 hiện là bản ổn định. Nó hỗ trợ viết, test, debug và deploy smart contract.

A.6. Cài MetaMask

1. Truy cập trang chính thức: MetaMask Download

2. Chọn Chrome/Edge/Firefox và cài extension từ cửa hàng extension chính thức của trình duyệt.

3. Tạo một ví TEST riêng cho môn học. Không sử dụng ví đang chứa tài sản thật.

4. Khi làm lab, thêm/kết nối local network tương ứng với Ganache và import một account test từ Ganache nếu cần.

A.7. Solidity – không cần cài riêng

Solidity compiler được Truffle quản lý. Cấu hình phiên bản compiler trong truffle-config.js:

```
compilers: {
   solc: {
      version: "0.5.2"
   }
}
```

A.8. Cài OpenZeppelin

```
cd tokenwallet
npm install openzeppelin-solidity@2.2.0
```

A.9. Tạo ReactJS project

ReactJS không cần tải một bộ cài riêng. Sau khi đã có Node.js/npm, chạy:

```
npx create-react-app tokenwallet
cd tokenwallet
npm install
```

Lệnh trên tạo cấu trúc ứng dụng React. Lưu ý create-react-app và các phiên bản React trong sách là stack cũ; mục tiêu ở đây là tái hiện project của Chương 2.

A.10. Cài Web3.js

Trong project React, nếu muốn bám theo phiên bản được sử dụng trong Chương 2:

```
npm install web3@1.2.0
```

Kiểm tra package đã xuất hiện trong package.json hoặc bằng:

```
npm list web3
```

A.11. Kiểm tra toàn bộ môi trường trước khi thực hành

Thành phần	Cách kiểm tra	Kết quả mong đợi
Node.js	node --version	Có số phiên bản
npm	npm --version	Có số phiên bản
Truffle	truffle version	Hiển thị Truffle/Solidity/Node
Ganache	Mở GUI	Có accounts và RPC server
MetaMask	Mở extension	Ví test hoạt động
OpenZeppelin	npm list openzeppelin-solidity	Hiển thị package
Web3.js	npm list web3	Hiển thị package